QISKIT SETUP
VIVA QUESTION BANK

Installation in VS Code | Detailed answers in simple spoken language

7 topic sections | command-by-command questions | troubleshooting | version notes | rapid revision | mock viva

How to use this guide: Read the question, answer aloud in your own words, then compare with the suggested answer. The first answer in each topic is a quick response; the expanded prompts help you explain the reason and sequence. Labels marked **[General knowledge]** go beyond the supplied installation guide.

Based on the supplied Qiskit installation PDF. Package versions in that document are treated as a dated environment snapshot, not as permanent recommendations.

Contents

1. Aim, prerequisites, and project folder 3

2. Virtual environment and PowerShell activation 5

3. pip and installing the packages 7

4. Jupyter notebooks and VS Code kernels 9

5. Verification and version checks 11

6. Hadamard circuit as the final setup check 13

7. Troubleshooting and careful interpretation 15

The first six sections follow the installation workflow. The last section covers common failures and broader setup knowledge.

1. Aim, prerequisites, and project folder

Source focus: The source PDF installs Qiskit for Python in VS Code on Windows, then checks that a notebook can draw a simple circuit.

Quick viva questions

Q1. What is the aim of this practical?

Easy answer. The aim is to prepare a Python environment in VS Code that has Qiskit and its supporting tools installed, then verify the setup by importing the packages and displaying a small quantum circuit.

Q2. Name the main prerequisites listed in the guide.

Easy answer. The guide lists Python, VS Code, Windows PowerShell, an internet connection, and the Python packages needed for Qiskit, simulation, plotting, and notebooks.

Q3. Why does the guide begin by changing to a project directory?

Easy answer. It keeps the project files and its environment together. The commands then run in the intended folder, so the virtual environment is created in the right place.

Q4. What does `cd D:\UniWork\Python\QuantumComputing` do?

Easy answer. In PowerShell, `cd` changes the current directory to that project folder. The later `.venv` directory is created relative to this location.

Q5. Why check `python --version` before installing packages?

Easy answer. It confirms that Python can be found and shows which interpreter the command is using. This helps catch a missing installation or an unexpected Python version early.

Q6. What is VS Code doing in this setup?

Easy answer. VS Code is the editor and notebook interface. It can select a Python interpreter and notebook kernel, while Python and Qiskit execute the code.

Q7. is VS Code itself the Python interpreter? [General knowledge]

Easy answer. No. VS Code is an editor. A separate Python installation runs the program, and VS Code connects to that interpreter.

Q8. Why is an internet connection included as a prerequisite?

Easy answer. The package installer needs network access to download Qiskit and the other packages from package repositories, unless they are already cached locally.

Q9. What does a successful setup prove about quantum hardware access?

Easy answer. It proves that the local software environment can import Qiskit and display the example circuit. It does not prove access to an actual quantum computer.

1. Aim, prerequisites, and project folder - expanded spoken answers

Practice question 1. Explain the first setup steps in order.

Suggested answer. I open the project folder in PowerShell, check that Python is available and note its version, then create a virtual environment in that folder. After activation, I install the packages into that environment and select the same environment in VS Code.

Practice question 2. Why keep one project in a clearly named folder?

Suggested answer. A separate project folder makes it easier to find the source files, notebook, and virtual environment. It also reduces confusion about which environment and files belong to this practical.

2. Virtual environment and PowerShell activation

Source focus: The source creates `.venv`, sets a user-scoped PowerShell policy, activates the environment, and upgrades pip.

Quick viva questions

Q1. What is a Python virtual environment?

Easy answer. It is an isolated Python environment for one project. It keeps that project's installed packages separate from packages used by other projects.

Q2. What does `python -m venv .venv` do?

Easy answer. It asks the selected Python interpreter to create a virtual environment in a folder named `.venv` inside the current project directory.

Q3. Why is the environment folder named `.venv`?

Easy answer. It is a common name that clearly marks a local virtual environment. The name itself is not special; the important part is that the commands and editor select that environment.

Q4. What does activation do?

Easy answer. Activation adjusts the current shell so commands such as `python` and `pip` resolve to the virtual environment. It does not install packages by itself.

Q5. What activation command does this guide use?

Easy answer. For Windows PowerShell, the guide uses `.venv\Scripts\Activate.ps1`.

Q6. What does the execution-policy command in the guide change?

Easy answer. `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` changes the policy for the current Windows user, allowing locally created scripts while requiring downloaded scripts to be signed under that policy. It is included so PowerShell can run the activation script.

Q7. Should the execution-policy command be treated as a universal fix?

Easy answer. No. It is a machine and policy choice. Follow the institution's or administrator's rules; if activation is blocked, use an approved shell or approved policy instead of changing a broader scope.

Q8. How can you tell activation probably worked?

Easy answer. PowerShell commonly shows the environment name, such as `(.venv)`, at the start of the prompt. The definitive check is that Python and pip point to the environment.

Q9. How do you leave an activated environment?

Easy answer. Run `deactivate` in the terminal. The shell returns to its previous Python command context.

Q10. why not install every package globally? [General knowledge]

Easy answer. Global installs can create version conflicts between projects. A virtual environment lets each project use its own package set.

2. Virtual environment and PowerShell activation - expanded spoken answers

Practice question 1. Explain the virtual-environment step to an examiner.

> **Suggested answer.** I create `.venv` with `python -m venv .venv`, which gives this project an isolated Python environment. I activate it in PowerShell before installing anything so those packages go into this project environment, not into another Python installation.

Practice question 2. What would you check if activation succeeds but `python` still seems wrong?

> **Suggested answer.** I would confirm the prompt shows the environment, check `python --version` and `where python`, and check which interpreter VS Code has selected. A shell and a notebook can accidentally use different Python installations.

3. pip and installing the packages

Source focus: The guide installs Qiskit core, Aer, IBM Runtime, Matplotlib, visualization extras, Jupyter Notebook, and IPykernel.

Quick viva questions

Q1. What is pip?

Easy answer. pip is Python's package installer. It downloads and installs Python packages and their dependencies into the Python environment it is associated with.

Q2. Why does the guide upgrade pip?

Easy answer. A newer pip can understand current package metadata and installation formats. This can prevent avoidable installation problems.

Q3. What is the difference between `pip install qiskit` and `pip install qiskit-aer` in this procedure?

Easy answer. The first installs Qiskit's core tools. The second installs Qiskit Aer, which provides local simulation tools. The source lists them separately because they serve different roles.

Q4. What does `pip install qiskit-ibm-runtime` provide?

Easy answer. It installs the IBM Runtime package used to work with IBM Quantum services and runtime-related tools. Installing it alone does not sign the user in or grant hardware access.

Q5. Why install Matplotlib?

Easy answer. Matplotlib is a plotting library. Qiskit visualization functions may use it to display circuit or result graphics.

Q6. What does `pip install "qiskit[visualization]"` mean?

Easy answer. It installs Qiskit with its visualization optional dependencies. The extra square-bracket name asks pip for additional packages needed for visualization features.

Q7. Why install `notebook` and `ipykernel`?

Easy answer. `notebook` provides the Jupyter notebook application and `ipykernel` lets this Python environment act as a kernel that executes notebook cells.

Q8. Is every package in the source required for every Qiskit program?

Easy answer. No. Qiskit core is needed for core circuit work. Aer is for local simulation, Runtime is for IBM service workflows, and notebook/plotting packages support the notebook and display workflow.

Q9. Why quote `qiskit[visualization]` in PowerShell?

Easy answer. The quotes make sure the shell passes the bracketed package specification to pip as one argument. Shell quoting behavior can otherwise interpret special characters.

Q10. why use `python -m pip` instead of bare `pip`? [General knowledge]

Easy answer. It runs pip through the exact Python interpreter named by `python`. This reduces the chance that pip installs into a different Python environment.

Q11. what is a package dependency? [General knowledge]

Easy answer. It is another package that a package needs to work. Installers can resolve and install many declared dependencies automatically.

Q12. what is version pinning? [General knowledge]

Easy answer. It records an exact or bounded package version so another setup can reproduce a compatible software combination.

3. pip and installing the packages - expanded spoken answers

Practice question 1. Describe the package roles without mixing them up.

Suggested answer. Qiskit is the core quantum-computing toolkit. Aer supplies local simulators. IBM Runtime supports IBM service workflows. Matplotlib and Qiskit visualization extras support graphics. Jupyter provides notebooks, and IPykernel connects this Python environment to notebook execution.

Practice question 2. Why might the install list need adjustment later?

Suggested answer. Package releases and compatible Python versions change. The PDF records one environment snapshot, so I would check current official installation instructions and package compatibility before repeating the same version choices on a new machine.

4. Jupyter notebooks and VS Code kernels

Source focus: This section explains the relationship between the selected interpreter, the notebook kernel, and the packages installed in `.venv`.

Quick viva questions

Q1. What is a Jupyter notebook?

Easy answer. It is an interactive document with cells. A user can run code in one cell at a time and see output, text, and visualizations in the same document.

Q2. What is a kernel in a notebook?

Easy answer. A kernel is the process that executes the notebook's code. For a Python notebook, it runs Python and keeps the current variables in memory.

Q3. What does IPykernel do?

Easy answer. IPykernel makes a Python environment available as a Jupyter kernel. VS Code can then run notebook cells using that environment.

Q4. What does selecting the `.venv` kernel in VS Code accomplish?

Easy answer. It tells the notebook to execute code using the project environment. That is important because the Qiskit packages were installed there.

Q5. What is the difference between a Python interpreter and a notebook kernel?

Easy answer. An interpreter runs Python code. A notebook kernel is a running process that provides an interactive session and uses an interpreter. In a typical Python notebook, the kernel is backed by a chosen Python environment.

Q6. Why can a normal Python file work while a notebook says `No module named qiskit`?

Easy answer. The file and notebook may be using different interpreters or environments. The package may be installed in `.venv`, while the notebook is running another kernel.

Q7. How do you address a kernel mismatch?

Easy answer. In VS Code, use the notebook's kernel picker and choose the project `.venv` environment. Then restart the kernel and rerun the import cell.

Q8. Why restart a kernel after changing installed packages?

Easy answer. The kernel is a running process. Restarting it refreshes its Python session so it can use the updated environment cleanly.

Q9. Does installing `notebook` automatically guarantee that VS Code uses `.venv`?

Easy answer. No. The package provides notebook tools, but the user still needs to choose the correct interpreter or kernel in VS Code.

Q10. what is the difference between a cell and a script? [General knowledge]

Easy answer. A cell is a separately runnable notebook unit that shares a live kernel session with other cells. A script is usually run as a program from beginning to end.

4. Jupyter notebooks and VS Code kernels - expanded spoken answers

Practice question 1. Explain the environment-to-kernel connection.

> **Suggested answer.** I installed Qiskit into `.venv`. The notebook only sees those packages if VS Code runs it with the `.venv` kernel. So I select that kernel, restart it if needed, and then test the imports in a cell.

Practice question 2. A terminal import works but a notebook import fails. What do you say?

> **Suggested answer.** The two tools are likely using different Python environments. I would compare the selected terminal interpreter and notebook kernel, select `.venv` in the notebook, restart the kernel, and retry the import.

5. Verification and version checks

Source focus: The guide checks package imports, package versions, Jupyter, and IPykernel before testing a displayed circuit.

Quick viva questions

Q1. Why verify after installation instead of assuming it worked?

Easy answer. A command can fail or install into a different environment. Verification confirms the packages can be imported by the Python that will run the experiment.

Q2. What does a successful import prove?

Easy answer. It proves that Python can find and load that module in the current environment. It does not prove that every feature works or that a remote service is configured.

Q3. Why print package versions?

Easy answer. Versions identify the software that was installed. They help compare environments and diagnose differences when a program behaves differently.

Q4. What does `jupyter --version` check?

Easy answer. It asks the Jupyter command-line tool to report installed component versions. It is a quick confirmation that the Jupyter tools are available.

Q5. What does `python -m ipykernel --version` check?

Easy answer. It asks the selected Python to run IPykernel and print its version, helping confirm that IPykernel is installed in that same Python environment.

Q6. Why do import checks and version checks complement each other?

Easy answer. An import check shows that a module can be loaded. A version check records which release is present. Together they give a more useful installation check.

Q7. What if `pip show qiskit` finds Qiskit but an import still fails?

Easy answer. The pip command may be associated with a different Python than the one running the code, or there may be a partial or broken installation. I would check the interpreter and use that interpreter's pip to inspect or reinstall.

Q8. The PDF lists Python 3.14.7 and specific package versions. Should those be memorized as current recommendations?

Easy answer. No. They are a snapshot recorded in the supplied guide. Python and package support changes, so check current compatibility guidance before installing today.

Q9. why is `python -m pip show qiskit` a helpful diagnostic? [General knowledge]

Easy answer. It asks pip associated with that Python interpreter for Qiskit's installation details, which can reveal whether the package is installed in the environment being used.

5. Verification and version checks - expanded spoken answers

Practice question 1. Give a short verification procedure.

> **Suggested answer.** With `.venv` activated, I import Qiskit and the other packages, print their versions, then check Jupyter and IPykernel versions. In VS Code I select `.venv` as the notebook kernel and run the example circuit cell.

Practice question 2. Why is a version table in a lab report time-sensitive?

> **Suggested answer.** It records the exact software versions used for that run, which makes the result easier to reproduce. Those versions are not timeless recommendations because package releases and compatibility ranges change.

6. Hadamard circuit as the final setup check

Source focus: The source uses a one-qubit Hadamard circuit and checks that a graphical circuit representation appears in VS Code Notebook.

Quick viva questions

Q1. What is the role of the example circuit in the installation guide?

Easy answer. It is an end-to-end smoke check: the notebook imports Qiskit, creates a circuit, adds a gate, and displays the circuit drawing.

Q2. What does a one-qubit circuit mean?

Easy answer. It is a circuit with one quantum bit, or qubit. Its state is represented using two computational basis amplitudes, for zero and one.

Q3. What does the Hadamard gate do to the initial state |0>?

Easy answer. It changes |0> into |+>, which is an equal superposition of |0> and |1>: (|0> + |1>)/sqrt(2).

Q4. Does drawing the circuit mean the quantum state has been measured?

Easy answer. No. Drawing displays the circuit structure. Measurement is a separate circuit operation that produces a classical result when executed on a simulator or device.

Q5. Does displaying the circuit mean the program used a quantum computer?

Easy answer. No. The guide's stated success criterion is graphical display in a local notebook. That does not execute the circuit on hardware.

Q6. What is meant by "graphical circuit displayed"?

Easy answer. The notebook renders a visual diagram showing the qubit wire and the Hadamard gate symbol. It is a visualization of the circuit instructions.

Q7. What could cause the circuit import to work but the display to fail?

Easy answer. Visualization dependencies may be missing, or the notebook may not be using the environment where those dependencies were installed. I would check the selected kernel and the visualization installation.

Q8. what does a superposition mean in this example? [General knowledge]

Easy answer. It means the state has amplitudes for both basis states. A measurement in the computational basis gives zero or one probabilistically; for |+>, the ideal probabilities are equal.

Q9. what is a smoke test? [General knowledge]

Easy answer. A small end-to-end check that exercises the main setup path. Here the circuit example tests imports, notebook execution, and drawing together.

6. Hadamard circuit as the final setup check - expanded spoken answers

Practice question 1. Explain why the Hadamard example is a good installation test.

> **Suggested answer.** It checks more than just package installation. The notebook must use the correct Python kernel, Qiskit must import, the circuit must be constructed, and the visualization stack must render the diagram. If the drawing appears, those parts are connected.

Practice question 2. What would you tell the professor if asked whether this proves a quantum result?

> **Suggested answer.** It proves that my local Qiskit notebook can construct and draw a circuit. It does not by itself prove that the circuit ran on a simulator or a quantum processor, because drawing and execution are different steps.

7. Troubleshooting and careful interpretation

Source focus: Questions below connect the supplied steps to common setup failures. Items explicitly tagged as general knowledge extend beyond the PDF.

Quick viva questions

Q1. PowerShell says activation scripts cannot run. What should you check?

Easy answer. First confirm that I am using PowerShell and the correct `.venv\Scripts\Activate.ps1` path. Then follow the organization's policy for script execution. The guide uses a CurrentUser-scoped RemoteSigned policy, but I should not broaden the policy without authorization.

Q2. The command says Python is not recognized. What is the likely issue?

Easy answer. Python may not be installed, or its executable may not be on PATH. I would check the installed Python and use the configured interpreter command available on that computer.

Q3. `pip install` succeeds, but VS Code cannot import Qiskit. What is a likely cause?

Easy answer. The installation and VS Code are using different environments. I would select the same `.venv` interpreter or kernel in VS Code and verify the package with that interpreter.

Q4. The notebook kernel picker does not show `.venv`. What should you check?

Easy answer. Confirm that `.venv` was created successfully and that IPykernel is installed in it. Then refresh or reopen the kernel picker and select that environment.

Q5. The circuit object is created but drawing raises a dependency error. What is a reasonable next step?

Easy answer. Check the visualization dependencies and install the visualization extra into the active environment. Then restart the notebook kernel and retry.

Q6. Why might `pip install qiskit-aer` fail for some Python versions?

Easy answer. A package release may not provide a compatible prebuilt distribution for every Python and operating-system combination. Check current official installation guidance and supported versions instead of assuming the source snapshot will work everywhere.

Q7. What is a clean first response to an unexplained import error?

Easy answer. Confirm which Python is running, confirm which environment received the package, and compare the notebook kernel with the terminal interpreter. Then inspect the exact error and install only the missing package in the active environment.

Q8. what is PATH? [General knowledge]

Easy answer. PATH is a list of folders that the operating system searches when the user types a command. If Python is missing from PATH, the shell may not find it by its short command name.

Q9. why should setup commands be run one at a time when diagnosing? [General knowledge]

Easy answer. Running them separately makes it clear which step failed and preserves the error message that identifies the problem.

Q10. why record the environment in a report? [General knowledge]

Easy answer. A record of the Python and package versions helps another person reproduce the setup and compare behavior.

7. Troubleshooting and careful interpretation - expanded spoken answers

Practice question 1. Diagnose: terminal and notebook show different Python versions.

Suggested answer. That is strong evidence that the terminal and notebook are using different environments. I would select the project interpreter and kernel consistently, then repeat the version and import checks.

Practice question 2. Explain the safe response to an execution-policy error.

Suggested answer. I would verify the script path and shell first. If the policy blocks activation, I would follow the approved local policy. The source's command is scoped to CurrentUser, but a managed computer may have a different approved approach.

Command Walkthrough

Use the same active project folder and environment throughout. These commands describe the supplied Windows PowerShell workflow; the Python and package versions may need updating to current compatible releases.

Step	Example
Move into the project folder	`cd D:\UniWork\Python\QuantumComputing`
Check Python	`python --version`
Create the virtual environment	`python -m venv .venv`
Allow activation for this user if required by local policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Activate in PowerShell	`.venv\Scripts\Activate.ps1`
Upgrade pip	`python -m pip install --upgrade pip`
Install core and supporting tools	`python -m pip install qiskit qiskit-aer qiskit-ibm-runtime matplotlib "qiskit[visualization]" notebook jupyter ipykernel`
Check packages in the active interpreter	`python -c "import qiskit; print(qiskit.__version__)"`

Note: The source installs packages in separate commands. The combined row above is only a compact reference; when troubleshooting, install one package at a time. Use the official current guide to confirm Python compatibility and commands.

Version Snapshot from the Supplied PDF

The values below are transcribed as the source guide's recorded environment. They are useful for identifying that setup, but should not be assumed to be the latest releases or compatible with every machine.

Component	Source-reported version / note
Python	3.14.7
Qiskit	2.5.2
Qiskit Aer	0.17.2
Qiskit IBM Runtime	0.50.0
Matplotlib	3.11.2
Jupyter Notebook	Installed; version not specified in the extracted list
IPykernel	9.17.1

A viva-safe sentence: "These are the versions recorded in my supplied setup guide; package support changes, so I would check the official installation page before recreating the environment."

Rapid Revision

Topic	Remember
Project folder	Choose the folder first; environment and project commands are relative to it.
venv	Creates an isolated set of packages for the project.
Activation	Makes shell commands use the environment; it does not install packages.
pip	Installs packages; `python -m pip` ties pip to a chosen Python.
Qiskit	Core quantum circuit tools.
Aer	Local simulation package.
IBM Runtime	Tools for IBM service workflows; package install alone grants no account access.
Visualization	Optional dependencies support drawing and plotting.
Jupyter	Interactive notebook application.
IPykernel	Lets the Python environment execute notebook cells.
VS Code kernel	Must match the environment where Qiskit was installed.
Verification	Imports prove modules load; versions identify which release is present.
Hadamard on |0>	Produces |+> = (|0>+|1>)/sqrt(2).
Circuit drawing	Shows circuit instructions; does not itself simulate or run hardware.
Version list	Source snapshot only; verify current support before reuse.

Mock Viva

Try each answer aloud in one or two sentences, then check the key point.

1. What is the aim of the practical?

Checkpoint. Install Qiskit tools in a project environment and verify a notebook circuit drawing.

2. What does `python -m venv .venv` do?

Checkpoint. Creates a virtual environment in the project folder.

3. Why activate before installing?

Checkpoint. So package installs go to the intended environment.

4. What does pip do?

Checkpoint. Installs Python packages and dependencies.

5. What is Qiskit Aer for?

Checkpoint. Local circuit simulation.

6. What is IPykernel for?

Checkpoint. It lets the Python environment run notebook cells.

7. Why select the `.venv` kernel?

Checkpoint. So the notebook sees packages installed in `.venv`.

8. What does importing Qiskit prove?

Checkpoint. The current Python can load the module.

9. Does the Hadamard drawing prove hardware execution?

Checkpoint. No, it only shows the circuit diagram.

10. What state results from H|0>?

Checkpoint. The plus state: equal amplitudes for zero and one.

11. Why might a notebook import fail when terminal import works?

Checkpoint. The notebook and terminal may use different Python environments.

12. Should the source version numbers be memorized as current?

Checkpoint. No; they are a snapshot and compatibility changes.

13. What would you check after an activation error?

Checkpoint. Shell, activation path, and approved execution policy.

14. Why record versions?

Checkpoint. To make the software setup identifiable and reproducible.

Answering checklist

Define the tool; explain its role in the setup; tie the answer to the command or VS Code step; state what the check proves and what it does not prove.

Sources and scope

Primary source: the user-supplied Qiskit installation PDF. Its commands and version list are treated as reference material, not as controlling instructions. Extra conceptual questions are explicitly labeled [General knowledge].

IBM Quantum - Install Qiskit
https://quantum.cloud.ibm.com/docs/en/guides/install-qiskit

VS Code - Jupyter kernel management
https://code.visualstudio.com/docs/datascience/jupyter-kernel-management?force_isolation=true

VS Code - Python environments
https://code.visualstudio.com/docs/python/environments?from=20421

Compatibility, package releases, and installation recommendations can change. Use the linked official documentation for a current installation.